Description Submission Düzenle Submission view

Due date: Cumartesi, 2 Mayıs 2026, 3:00 AM
Requested files: Biography.java, Book.java, Library.java, Main.java, Novel.java, TextBook.java (Download)
Type of work: Individual work

Book Library Manager with Inheritance, Map and Set

Goal

In this lab you will use three topics together:

1. **OOP, Inheritance and Polymorphism:** an `abstract` parent class + child classes
2. **`Map`:** fast look-up from ID to book + a type index (Map-of-Set)
3. **`Set`:** any time you need a group of items without duplicates (read IDs, IDs under a type, and the book groups that listing methods give back)

The Story

You will write a program that keeps books in a library and groups them by type. There will be **three book types**:

Child Class	Extra Field	Type
`Novel`	`pageCount`	`int`
`Biography`	`autobiography` (is it written by the person themselves?)	`boolean`
`TextBook`	`subject` (for example "Math", "History", "Physics")	`String`

Every book has these shared fields: `id`, `title`, `author`, `year`, `rating`.

Class Structure

The code uses one parent class and three child classes.

`Book` is the **parent** class. It is declared as `abstract`, which means you cannot build a `Book` object with `new Book(...)`. It only holds the things every book has in common: the `id`, `title`, `author`, `year` and `rating` fields, plus a placeholder method called `getType()` that has no body yet.

`Novel`, `Biography`, and `TextBook` are the **child** classes. Each of them `extends Book`. A child class inherits the shared fields from the parent and adds one extra field of its own (a page count, an autobiography flag, or a subject). Each child also fills in the empty `getType()` with its own answer (`"Novel"`, `"Biography"`, or `"TextBook"`) and adds its extra piece at the end of the line that `toString()` returns.

Because every child `is a Book`, any of them can be stored in a variable of type `Book`. That is why `Library` can keep all three types together inside a single `Map<String, Book>`.

Data Structure

Inside `Library` you will keep **three collections**:

Field	Type	Goal
`books`	`Map<String, Book>`	ID → Book object
`readBooks`	`Set<String>`	IDs of read books (no duplicates)
`typeIndex`	`Map<String, Set<String>>`	Type name → Set of IDs under that type

Imagine that four books have been added (B001 Novel, B002 Biography, B003 TextBook, B004 TextBook) and B001 + B003 have been marked as read. The three collections now look like this in memory.

`books` holds every book. The key is the ID, the value is the whole `Book` object:

```
"B001" -> 1984 (Novel)
"B002" -> Diary of a Young Girl (Biography)
"B003" -> Calculus Made Easy (TextBook)
"B004" -> A Brief History of Time (TextBook)
```

`readBooks` is a flat `Set` of IDs, nothing more. If an ID is inside this Set, that book counts as read:

```
{"B001", "B003"}
```

`typeIndex` is a **Map of Sets**. The outer `Map` key is a type name, and each value is an inner `Set` of the IDs of the books that belong to that type:

```
"Novel"     -> {"B001"}
"Biography" -> {"B002"}
"TextBook"  -> {"B003", "B004"}
```

When a brand new type comes in for the first time (say `"Comics"`), `typeIndex` has no entry for it yet. You have to create a new empty Set and put it under that type **before** you can add an ID into it.

Files

Given to you (do not change these)

- `Main.java`: runs test scenarios and prints the results.

You will fill in (signatures are ready, write the bodies)

1. `Book.java`: abstract parent class
2. `Novel.java`: extends Book
3. `Biography.java`: extends Book
4. `TextBook.java`: extends Book
5. `Library.java`: handles the collection

To Do: What is Expected From You

For every method below, the signature is already written. Your job is to make the method behave the way it is described. You decide how to write the code.

Step 1: `Book.java` (abstract)

Make the class behave like this:

- **Constructor**: When a `Book` object is created, the object must remember all the information passed in, so the getters can hand it back later.
- **Getters**: Each getter gives back the value of its matching field.
- `getType()`: This method has no body here because `Book` is abstract. Every child class gives the real answer.
- `toString()`: Must give back a single line that fully describes the book. The line must contain the ID, the title, the year, the author, the rating (shown with one decimal place), and the type name. Example output:

  ```
  [ID: B001] 1984 (1949) - George Orwell | Rating: 9.0 | Type: Novel
  ```

  Tip: use `String.format` and call `getType()` inside it. Thanks to polymorphism, the correct child type name shows up for each book, even though the format lives in the parent class.

Step 2: Child Classes (`Novel`, `Biography`, `TextBook`)

Each child class follows the same idea. What is expected:

- **Constructor**: Creates a book of this specific type. Remembers both the shared fields (through the parent constructor) and the type's own extra field.
- **Getter for the extra field**: Gives back that extra field.
- `getType()`: Gives back the type name as a plain String. `Novel` returns `"Novel"`, `Biography` returns `"Biography"`, `TextBook` returns `"TextBook"`.
- `toString()`: Gives back the parent's description plus this type's extra information at the end of the same line. The extras are:
  - `Novel` adds `" | Pages: <number>"`
  - `Biography` adds `" | Autobiography: <true or false>"`
  - `TextBook` adds `" | Subject: <subject>"`

Step 3: `Library.java`

The three fields are already defined for you:

```
private Map<String, Book> books = new HashMap<>();
private Set<String> readBooks = new HashSet<>();
private Map<String, Set<String>> typeIndex = new HashMap<>();
```

Every method reads from or writes to these three fields. Nothing is printed inside `Library`, each method gives back a value.

Method	What is expected
`boolean addBook(Book b)`	The new book must land in **two** places at the same time. **(1)** In the `books` Map, stored with the book's ID as the key. **(2)** In `typeIndex`, under the book's type name. The ID goes into the inner Set that sits at that key (see the `typeIndex` picture in the Data Structure section). If no Set exists for this type yet, first create a new empty Set, put it into `typeIndex` under the type name, and then add the ID into that Set. If the ID is already in `books`, nothing changes anywhere and the method gives back `false`. On a fresh add, gives back `true`.
`Book findBook(String id)`	Looks up the ID in the `books` Map and gives back the matching book. Gives back `null` when no book has that ID.
`boolean markAsRead(String id)`	Puts the ID into the `readBooks` Set. Gives back `true` only when a new ID is actually added. Gives back `false` when the book does not exist in the `books` Map, or when the ID was already in the `readBooks` Set.
`boolean markAsUnread(String id)`	Takes the ID out of the `readBooks` Set. Gives back `true` when an ID is actually removed. Gives back `false` when the ID was not in the Set.
`boolean isRead(String id)`	Gives back `true` when the ID is inside the `readBooks` Set, `false` otherwise.
`Set<Book> listAllBooks()`	Gives back a Set built from every value in the `books` Map. Empty set when `books` is empty.
`Set<Book> listReadBooks()`	Walks through the IDs in the `readBooks` Set and collects their matching books from the `books` Map into a new Set. Empty set when nothing is read.
`Set<Book> listUnreadBooks()`	Collects every book from the `books` Map whose ID is **not** in the `readBooks` Set. Empty set when every book is read.
`Set<Book> listBooksByType(String type)`	Looks up the inner Set of IDs under this type in `typeIndex`, then collects the matching books from the `books` Map into a new Set. Empty set when that type has no entry in `typeIndex` or holds no IDs.
`Map<String, Integer> typeStatistics()`	Builds a new Map where each key is a type name from `typeIndex` and each value is the size of that type's inner Set. Example: `{"Novel"=1, "Biography"=1, "TextBook"=2}`. Empty map when `typeIndex` is empty.

Tips

- You cannot create an object from an abstract class: `new Book(...)` gives an error, so every object stored in `Map<String, Book>` is really a child class object. That is polymorphism.
- `super.toString()` lets you reach the parent class's `toString`.
- Remember to create the inner Set first when `typeIndex` does not yet have the type key, or you will get a `NullPointerException`.
- When a set method has nothing to give back, return an empty set (`new HashSet<>()`), not `null`. That way `Main` can always loop over the result safely.
- Do **not** change the method signatures. The tests in `Main.java` use them as they are.

Expected output

```
===== TEST 1: Add books =====
add B001 Novel:     true
add B002 Biography: true
add B003 TextBook:  true
add B004 TextBook:  true

===== TEST 2: Duplicate ID =====
add B001 again:     false

===== TEST 3: Mark as read =====
mark B001:          true
mark B003:          true
mark B001 again:    false
mark B999:          false

===== TEST 4: isRead checks =====
isRead B001: true
isRead B002: false
isRead B999: false

===== TEST 5: List all books =====
count: 4
[ID: B003] Calculus Made Easy (1910) - Silvanus Thompson | Rating: 7,8 | Type: TextBook | Subject: Math [READ]
[ID: B001] 1984 (1949) - George Orwell | Rating: 9,0 | Type: Novel | Pages: 328 [READ]
[ID: B002] Diary of a Young Girl (1947) - Anne Frank | Rating: 8,5 | Type: Biography | Autobiography: true
[ID: B004] A Brief History of Time (1988) - Stephen Hawking | Rating: 8,7 | Type: TextBook | Subject: Physics

===== TEST 6: Find book by ID =====
find B002: [ID: B002] Diary of a Young Girl (1947) - Anne Frank | Rating: 8,5 | Type: Biography | Autobiography: true
find B999: null

===== TEST 7: Read books =====
count: 2
[ID: B003] Calculus Made Easy (1910) - Silvanus Thompson | Rating: 7,8 | Type: TextBook | Subject: Math
[ID: B001] 1984 (1949) - George Orwell | Rating: 9,0 | Type: Novel | Pages: 328

===== TEST 8: Unread books =====
count: 2
[ID: B002] Diary of a Young Girl (1947) - Anne Frank | Rating: 8,5 | Type: Biography | Autobiography: true
[ID: B004] A Brief History of Time (1988) - Stephen Hawking | Rating: 8,7 | Type: TextBook | Subject: Physics

===== TEST 9: Remove read mark =====
unmark B001:        true
unmark B001 again:  false

===== TEST 10: Books of type TextBook =====
count: 2
[ID: B003] Calculus Made Easy (1910) - Silvanus Thompson | Rating: 7,8 | Type: TextBook | Subject: Math
[ID: B004] A Brief History of Time (1988) - Stephen Hawking | Rating: 8,7 | Type: TextBook | Subject: Physics

===== TEST 11: Books of type Comics =====
count: 0

===== TEST 12: Type statistics =====
Novel:     1
Biography: 1
TextBook:  2
```

Note: in the listings, the order of books may look different from this example because `HashMap` and `HashSet` do not keep insertion order. That is normal.

Your Code Must Compile

The minimum bar for a valid submission is that your code **compiles**. Once you hand it in, **no changes will be made to your code** for any reason. Excuses like "I forgot a `;`" or "I missed one line" will not be accepted. Code that does not compile gets **0 points**, no matter how close to finished it looked.

Managing your time so that you hand in a code that compiles is your own responsibility. Failing to compile is not a small problem. It is one of the most critical mistakes you can make, because it means a product that does not run at all from the very first moment.

Requested files

Biography.java

```java
public class Biography extends Book {
    private boolean autobiography;

    public Biography(String id, String title, String author, int year,
                     double rating, boolean autobiography) {
        super(id, title, author, year, rating);
        // TODO: This object must also remember whether the book is an autobiography.
    }

    // For boolean fields we usually start the getter with "is..." instead of "get..."
    public boolean isAutobiography() {
        // TODO: Give back the autobiography value.
        return false;
    }

    @Override
    public String getType() {
        // TODO: Give back this book's type name.
        return null;
    }

    @Override
    public String toString() {
        // TODO: Give back the parent's description plus this type's extra info
        //       at the end of the same line, like " | Autobiography: true".
        return null;
    }
}

```

Book.java

```java
public abstract class Book {
    protected String id;
    protected String title;
    protected String author;
    protected int year;
    protected double rating;

    public Book(String id, String title, String author, int year, double rating) {
        // TODO: When a Book object is created, it must remember the given
        //       information so the getters can hand it back later.
    }

    public String getId() {
        // TODO: Give back the book's id.
        return null;
    }

    public String getTitle() {
        // TODO: Give back the book's title.
        return null;
    }

    public String getAuthor() {
        // TODO: Give back the book's author.
        return null;
    }

    public int getYear() {
        // TODO: Give back the book's year.
        return 0;
    }

    public double getRating() {
        // TODO: Give back the book's rating.
        return 0.0;
    }

    // This method has no body because the class is abstract.
    // Every child class MUST override it and return its own type name.
    public abstract String getType();

    @Override
    public String toString() {
        // TODO: Give back a single line that fully describes this book.
        //       The line must show: id, title, year, author, rating (one decimal place)
        //       and the type name.
        //       Example output:
        //         [ID: B001] 1984 (1949) - George Orwell | Rating: 9.0 | Type: Novel
        return null;
    }
}

```

Library.java

```java
import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;
import java.util.Set;

public class Library {
    private Map<String, Book> books = new HashMap<>();
    private Set<String> readBooks = new HashSet<>();
    private Map<String, Set<String>> typeIndex = new HashMap<>();

    public boolean addBook(Book b) {
        // TODO: Add the book to the collection and keep typeIndex in sync
        //       so that later queries by type can find this book.
        //       If another book already has the same ID, do NOT add it.
        //       Give back true when the book is added, false when the ID is a duplicate.
        return false;
    }

    public Book findBook(String id) {
        // TODO: Give back the book with this ID. Give back null if no book has it.
        return null;
    }

    public boolean markAsRead(String id) {
        // TODO: Mark the book as read.
        //       Give back true only when the mark was newly added.
        //       Give back false if the book does not exist OR it was already read.
        return false;
    }

    public boolean markAsUnread(String id) {
        // TODO: Remove the read mark from the book.
        //       Give back true when a mark was actually removed.
        //       Give back false when the book was not marked as read.
        return false;
    }

    public boolean isRead(String id) {
        // TODO: Give back true if the book with this ID is currently marked
        //       as read, false otherwise.
        return false;
    }

    public Set<Book> listAllBooks() {
        // TODO: Give back a Set that contains every book in the collection.
        //       If the collection is empty, give back an empty set.
        return new HashSet<>();
    }

    public Set<Book> listReadBooks() {
        // TODO: Give back a Set that contains only the read books.
        //       If nothing is read, give back an empty set.
        return new HashSet<>();
    }

    public Set<Book> listUnreadBooks() {
        // TODO: Give back a Set that contains only the books that are NOT read.
        //       If every book is read, give back an empty set.
        return new HashSet<>();
    }

    public Set<Book> listBooksByType(String type) {
        // TODO: Give back a Set that contains every book of the given type.
        //       If no book exists for that type, give back an empty set.
        return new HashSet<>();
    }

    public Map<String, Integer> typeStatistics() {
        // TODO: Give back a Map that pairs every type name with the number of
        //       books in that type. Example: {"Novel"=1, "Biography"=1, "TextBook"=2}.
        //       If the collection is empty, give back an empty map.
        return new HashMap<>();
    }
}

```

Main.java

```java
import java.util.Map;
import java.util.Set;

public class Main {
    public static void main(String[] args) {
        Library library = new Library();
        runTests(library);
    }

    // This method runs several test scenarios. Do not change it.
    // When your classes are ready, run Main and check the printed output.
    static void runTests(Library library) {

        System.out.println("===== TEST 1: Add books =====");
        System.out.println("add B001 Novel:     "
                + library.addBook(new Novel("B001", "1984", "George Orwell", 1949, 9.0, 328)));
        System.out.println("add B002 Biography: "
                + library.addBook(new Biography("B002", "Diary of a Young Girl", "Anne Frank", 1947, 8.5, true)));
        System.out.println("add B003 TextBook:  "
                + library.addBook(new TextBook("B003", "Calculus Made Easy", "Silvanus Thompson", 1910, 7.8, "Math")));
        System.out.println("add B004 TextBook:  "
                + library.addBook(new TextBook("B004", "A Brief History of Time", "Stephen Hawking", 1988, 8.7, "Physics")));

        System.out.println("\n===== TEST 2: Duplicate ID =====");
        System.out.println("add B001 again:     "
                + library.addBook(new Novel("B001", "Copy Book", "Nobody", 2020, 1.0, 0)));

        System.out.println("\n===== TEST 3: Mark as read =====");
        System.out.println("mark B001:          " + library.markAsRead("B001"));
        System.out.println("mark B003:          " + library.markAsRead("B003"));
        System.out.println("mark B001 again:    " + library.markAsRead("B001"));
        System.out.println("mark B999:          " + library.markAsRead("B999"));

        System.out.println("\n===== TEST 4: isRead checks =====");
        System.out.println("isRead B001: " + library.isRead("B001"));
        System.out.println("isRead B002: " + library.isRead("B002"));
        System.out.println("isRead B999: " + library.isRead("B999"));

        System.out.println("\n===== TEST 5: List all books =====");
        Set<Book> all = library.listAllBooks();
        System.out.println("count: " + all.size());
        for (Book b : all) {
            String label = library.isRead(b.getId()) ? " [READ]" : "";
            System.out.println(b + label);
        }

        System.out.println("\n===== TEST 6: Find book by ID =====");
        System.out.println("find B002: " + library.findBook("B002"));
        System.out.println("find B999: " + library.findBook("B999"));

        System.out.println("\n===== TEST 7: Read books =====");
        Set<Book> readSet = library.listReadBooks();
        System.out.println("count: " + readSet.size());
        for (Book b : readSet) System.out.println(b);

        System.out.println("\n===== TEST 8: Unread books =====");
        Set<Book> unread = library.listUnreadBooks();
        System.out.println("count: " + unread.size());
        for (Book b : unread) System.out.println(b);

        System.out.println("\n===== TEST 9: Remove read mark =====");
        System.out.println("unmark B001:        " + library.markAsUnread("B001"));
        System.out.println("unmark B001 again:  " + library.markAsUnread("B001"));

        System.out.println("\n===== TEST 10: Books of type TextBook =====");
        Set<Book> textbooks = library.listBooksByType("TextBook");
        System.out.println("count: " + textbooks.size());
        for (Book b : textbooks) System.out.println(b);

        System.out.println("\n===== TEST 11: Books of type Comics =====");
        Set<Book> comics = library.listBooksByType("Comics");
        System.out.println("count: " + comics.size());

        System.out.println("\n===== TEST 12: Type statistics =====");
        Map<String, Integer> stats = library.typeStatistics();
        System.out.println("Novel:     " + stats.get("Novel"));
        System.out.println("Biography: " + stats.get("Biography"));
        System.out.println("TextBook:  " + stats.get("TextBook"));
    }
}
```

Novel.java

```java
public class Novel extends Book {
    private int pageCount;

    public Novel(String id, String title, String author, int year,
                 double rating, int pageCount) {
        super(id, title, author, year, rating);
        // TODO: This object must also remember how many pages this novel has.
    }

    public int getPageCount() {
        // TODO: Give back the page count.
        return 0;
    }

    @Override
    public String getType() {
        // TODO: Give back this book's type name.
        return null;
    }

    @Override
    public String toString() {
        // TODO: Give back the parent's description plus this type's extra info
        //       at the end of the same line, like " | Pages: 328".
        return null;
    }
}

```

TextBook.java

```java
public class TextBook extends Book {
    private String subject;

    public TextBook(String id, String title, String author, int year,
                    double rating, String subject) {
        super(id, title, author, year, rating);
        // TODO: This object must also remember the subject of this textbook.
    }

    public String getSubject() {
        // TODO: Give back the subject.
        return null;
    }

    @Override
    public String getType() {
        // TODO: Give back this book's type name.
        return null;
    }

    @Override
    public String toString() {
        // TODO: Give back the parent's description plus this type's extra info
        //       at the end of the same line, like " | Subject: Math".
        return null;
    }
}

```

VPL